
DSC 140A - Super Homework

Due: Wednesday, March 19

Instructions: Write your solutions to the following problems either by typing them or handwriting them on another piece of paper or on an iPad/tablet. Show your work or provide justification unless otherwise noted; submissions that don't show work might lose credit. If you write code to solve a problem, include the code by copy/pasting or as a screenshot. You may use `numpy`, `pandas`, `matplotlib` (or another plotting library), and any standard library module, but no other third-party libraries unless specified. Submit homeworks via Gradescope by 11:59 PM.

A $\text{\LaTeX}$ template is provided at <http://dsc140a.com>, next to where you found this homework. Using it is totally optional, but encouraged if you plan to go to grad school. See this video for a quick introduction to $\text{\LaTeX}$.

Note: you *may* use a slip day on this assignment, if you have any remaining.

Programming Problem 1. (Classifier Competition)

In this problem, you'll use what you've learned this quarter to build a simple classifier that can tell the difference between Spanish and French. To make things interesting, we'll run a competition to see who can build the best classifier, extra credit being given for particularly accurate classifiers.

The problem. Consider the words "meilleur" and "mejor". In English, both of these words mean "best" – one in French and the other in Spanish. Even if you don't know how to speak either of these languages, you might be able to guess which word is Spanish and which is French based on the spelling. For example, the word "meilleur" *looks* more like a French word than a Spanish word due to it containing "ei" and ending in "eur". On the other hand, the word "mejor" *looks* more like a Spanish word than a French word due to it containing "j" and ending in "or". This suggests that there is some statistical structure in the words of these languages that we can use to build a machine learning model capable of distinguishing between French and Spanish without actually understanding the words themselves.

Your goal in this problem is to build a machine learning model that can take a word as input and predict whether it is Spanish or French.

What you'll turn in. Unlike other problems from this quarter, this problem will require you to turn in a code file. You will turn into Gradescope a Python file named `classify.py` containing a function `classify()` that takes three arguments:

- **train_words:** a list of n strings, each one of them a word (in either Spanish or French)
- **train_labels:** a list of n strings, each one of them either `"spanish"` or `"french"`, indicating the language of the corresponding word in **train_words**
- **test_words:** a list of m strings, each one of them a word (in either Spanish or French)

This function should return a list of m strings, each one of them either `"spanish"` or `"french"`, indicating your classifier's prediction for the language of the corresponding word in **test_words**.

Your `classify()` function is responsible for training your machine learning model on the training data and then using that model to make predictions on the test data.

You may include other helper functions in your `classify.py` file if you wish, but the only function that will be called by the autograder is `classify()`. You can import `numpy`, `pandas`, `scipy`, or any module in the

Python standard library, but you may *not* use any other third-party libraries (like `sklearn`). That is, you will need to implement your classifier “from scratch”.

The data. The data set linked below contains 1200 words, about half of which are Spanish and half of which are French:

https://f000.backblazeb2.com/file/jeldridge-data/012-spanish_french/train.csv

We will use this data set to train your model. It was constructed by translating common texts from English to Spanish and French and then replacing any accented characters with their unaccented equivalents and removing special characters.

Your model will be tested on a separate, unseen data set that was constructed in a similar way. The test data will be (approximately) balanced between Spanish and French words.

Rules.

- The code that you turn in can use `numpy`, `pandas`, `scipy`, or any module in the Python standard library, but you may *not* use any other third-party libraries (like `sklearn`). You will need to implement your classifier “from scratch”.
- The machine learning model that you implement must be one that we covered this quarter. Note that we did *not* cover logistic regression, so it is *not* a valid model!
- Your code should not use any data other than the training data passed to `classify()` to build the model. For example, you can’t download a Spanish dictionary from the internet and use it to make predictions!

Competition and Grading You will achieve full credit on this problem if your classifier satisfies the above rules and achieves an accuracy of at least 75% on the unseen test data set. We’ve tried a few simple models and found that it’s possible to achieve this level of accuracy with a relatively basic technique, so you will not need to do anything too fancy to achieve full credit.

If your classifier achieves an accuracy of less than 75%, you can receive partial credit.

If your classifier does *better* than 80% accuracy on the test set, you’ll get extra credit. More precisely, we’ll award 1 point of extra credit for every percentage point by which your classifier’s accuracy exceeds 80%. For example, if your classifier achieves 85% test accuracy, you’ll receive 5 points of extra credit. These extra credit points will be applied to your overall homework score for the quarter (not to the Super Homework itself) and can therefore earn credit back for points you might have missed on Homeworks 1-8. Your total homework credit for the quarter *will* be allowed to exceed 100%.

The winner(s) of the competition will be the student(s) who achieve the highest test accuracy. The winner(s) will not receive any additional extra credit on top of the extra credit that they receive for their test accuracy (they probably won’t need it!). However, they will receive a paragraph in any future letter of recommendation that I write for them describing their achievement.

As in the “real world”, the autograder will not tell you your test accuracy – you will need to estimate it yourself using the data that we provided. However, to give you some peace of mind, it will warn you if your test accuracy is below 75% (though it won’t tell you by how much).

Tips and Hints.

- A good choice of features is important. You might want to consider using the frequency of different letters or pairs of letters in each word as features. For example, one of your features might be whether “el” appears in the word. You do not need to create all of these features by hand – you can use Python to help you generate them.

- Don't confuse training accuracy with test accuracy. It is possible to achieve 90%+ training accuracy on this data set, but that doesn't mean your model will generalize well to the test set.
- Be careful to avoid overfitting! If you use too many features or too complex of a model, you may find that your model performs well on the training data but poorly on the test data.
- Start with the simplest models first. We have learned some models in this class that can be implemented using only a couple lines of code (with `numpy`).

Problem 1.

For this problem, you will need the data set below. It contains 50 points in $\mathbb{R}^2$ suitable for linear regression:

<https://f000.backblazeb2.com/file/jeldridge-data/013-regression/data.csv>

- a) Suppose we wish to fit a linear model $H(x) = w_0 + w_1x$ to this data, and it is known that $w_0 = 2$. Write a function, `mse(w_1)`, that takes in a possible value of w_1 and returns the mean squared error of the linear model $H(x) = 2 + w_1x$ with respect to the data set. You may use `numpy`. This problem is not autograded, but turn in your code (either copy/pasting or as a screenshot).

Hint: The MSE at $w_1 = 1$ should be approximately 139.6.

- b) Suppose we are now thinking probabilistically, and we believe that the data has been generated by the model $Y = w_0 + w_1X + \epsilon$, where $\epsilon \sim \mathcal{N}(0, \sigma^2)$. Suppose it is known that $\sigma = 4$ and that $w_0 = 2$.

Write a function, `log_likelihood(w_1)`, that takes in a possible value of w_1 and returns the log likelihood of the linear model $Y = 2 + w_1X + \epsilon$ with respect to the data set. You may use `numpy`. Show your code.

Hint: The log likelihood at $w_1 = 1$ should be approximately -333.39.

- c) Using `scipy.optimize.fmin`, find the value of w_1 that minimizes the mean squared error.

Note: doing this numerically is overkill: we have a closed-form solution for the value of w_1 that minimizes the mean squared error.

- d) Again using `scipy.optimize.fmin`, find the value of w_1 that maximizes the log likelihood.

- e) Plot the MSE for all values of w_1 between 0 and 5. On the same graph, plot the log likelihood for all of the same values. Plot a vertical line at the value of w_1 that minimizes the mean squared error (and maximize the log likelihood). Include a legend in your plot that tells which line is which.

Note: you may use `matplotlib` to make your plot. You might want to use `plt.axvline` to plot the vertical line and `plt.legend` to include a legend.

Problem 2.

Suppose you are training a decision tree classifier to predict whether it will rain in the next hour or not. To do so, you will use two features: the current temperature (in Fahrenheit) and the current pressure (in millibars). Here is some training data that you have collected:

Temperature	Pressure	Rain?
65	1001	Yes
72	1003	Yes
79	1030	No
55	1022	Yes
62	1025	No
71	1010	Yes
73	1011	No

Train the decision tree to a depth of two. In other words, decide on a root question, splitting the data into two groups, then decide on another question for each group (if necessary). Your resulting decision tree should have two interior (question) nodes and three leaf nodes and should classify all of the training data correctly. Use the Gini coefficient to measure uncertainty. Every time you must decide on a question, choose the one that minimizes the uncertainty of the resulting split. Use as your questions thresholds of the form: “Is temp. < 55?”, “Is temp. < 65?”, ..., “Is pressure < 1030?”.

You may solve this problem by hand, or you may write code to help you solve it. If you use code, you *may not* use any third party libraries besides `numpy` and `pandas`. If you use code, turn in your code with your solution.

Your answer should include:

- A drawing of the resulting decision tree that labels each interior node with the question that it asks and each leaf node with the class that it predicts.
- For each interior node, a calculation of the uncertainty of all possible splits, with the best split highlighted in some way.

Note: you should choose the *best split* at every step. Namely, your root node should ask the question that minimizes uncertainty. It is possible to choose a different root question and still get a decision tree of depth 2 that perfectly classifies the training data, but this tree will not be considered correct because there is a unique root question minimizing uncertainty.

Solution: Recall, at each node we choose a ‘question’ or *split* that results in the lowest uncertainty. A split on a set S of labeled points will form left- and right-sets, S_L, S_R , as well as the proportion of points in S that fell into each side, p_L, p_R . The uncertainty of a given split is calculated as the weighted sum of uncertainty of each of the resulting sets. Define the following

$$u(S) = 2p(1 - p) = 2(1 - p)p$$

$$u(\text{split on } S) = p_L u(S_L) + p_R u(S_R)$$

Our root question will examine full data set. For fun, we can calculate this initial uncertainty as

$$u(S) = 2 \cdot \frac{3}{7} \cdot \frac{4}{7} = 0.490$$

We can calculate uncertainty of possible splits, starting with the lowest to highest x_1 , then the lowest to highest x_2 , where x_1, x_2 are the Temperature and Pressure features. We will calculate the first three below, and the rest using code.

Notation: The symbol $:$ is often used in set notation, you can read it as meaning “such that”.

$$\begin{aligned}
u(x_1 < 55 ?) &\Rightarrow \text{No } [3, 4], \text{ Yes } [0, 0] \\
&= u(S : x_1 \not< 55) + 0 \\
&= u(S) \\
&= 0.490
\end{aligned}$$

$$\begin{aligned}
u(x_1 < 62 ?) &\Rightarrow \text{No } [3, 3], \text{ Yes } [0, 1] \\
&= \frac{6}{7}u(S : x_1 \not< 62) + \frac{1}{7}u(S : x_1 < 62) \\
&= \frac{6}{7}(2 \cdot \frac{1}{2} \cdot \frac{1}{2}) + \frac{1}{7}(2 \cdot 1 \cdot 0) \\
&= 0.429
\end{aligned}$$

$$\begin{aligned}
u(x_1 < 65 ?) &\Rightarrow \text{No } [2, 3], \text{ Yes } [1, 1] \\
&= \frac{5}{7}u(S : x_1 \not< 65) + \frac{2}{7}u(S : x_1 < 65) \\
&= \frac{5}{7}(2 \cdot \frac{2}{5} \cdot \frac{3}{5}) + \frac{2}{7}(2 \cdot \frac{1}{2} \cdot \frac{1}{2}) \\
&= 0.486
\end{aligned}$$

$$u(x_1 < 71 ?) \Rightarrow \dots$$

Once performed over all possible splits on S , we arrive at the minimum uncertainty with the split $x_2 < 1011$, with

$$\begin{aligned}
u(x_2 < 1011 ?) &\Rightarrow \text{No } [3, 1], \text{ Yes } [0, 3] \\
&= 0.214
\end{aligned}$$

Note that uncertainty is zero at our node $\{S : x_2 < 1011\}$, thus we consider that node a leaf. However we must continue reducing uncertainty for our node $\{S : x_2 \not< 1011\}$. To do this, we repeat the above process, but examining only this subset of data.

Since the current node was split using x_2 , it is wise to examine splits using x_1 . The first two are calculated below.

$$\begin{aligned}
u(x_1 < 55 ? : x_2 \not< 1011) &= u(S : x_2 \not< 1011, x_1 \not< 55) + 0 \\
&= u(S : x_2 \not< 1011) \\
&= 2 \cdot \frac{3}{4} \cdot \frac{1}{4} \\
&= 0.375
\end{aligned}$$

$$\begin{aligned}
u(x_1 < 62 ? : x_2 \not< 1011) &\Rightarrow \text{No } [0, 1], \text{ Yes } [3, 0] \\
&= \frac{1}{4}u(S : x_2 \not< 1011, x_1 \not< 62) + \frac{3}{4}u(S : x_2 \not< 1011, x_1 < 62) \\
&= \frac{1}{4}(2 \cdot 0 \cdot 1) + \frac{3}{4}(2 \cdot 1 \cdot 0) \\
&= 0
\end{aligned}$$

Aha! We've found another split that results in zero uncertainty. We can now consider the nodes $\{S : x_2 \not< 1011, x_1 < 62\}$ and $\{S : x_2 \not< 1011, x_1 \not< 62\}$ to be leaves!

Our final decision tree structure looks like

